

Практическое задание по теме «Задачи линейного программирования»

Выполнила: Данилова Айаана Васильевна

Вариант: 10

Формулировка задания по варианту:

Минимизировать $Z = 4x_1 + x_2 + x_3 + 2x_4$
при условиях:

$$2x_1 + x_2 + x_4 \leq 9$$

$$x_1 + x_2 + x_3 = 7$$

$$x_3 + x_4 \geq 5$$

Ход решения:

$$W(x) = 4x_1 + x_2 + x_3 + 2x_4 \rightarrow \min$$

$$\begin{cases} 2x_1 + x_2 + x_4 \leq 9 \\ x_1 + x_2 + x_3 = 7 \\ x_3 + x_4 \geq 5 \\ x_1, x_2, x_3, x_4 \geq 0 \end{cases}$$

1) Приведём систему к КЗЛП.

$$\begin{cases} 2x_1 + x_2 + x_4 + x_5 = 9 \\ x_1 + x_2 + x_3 = 7 \\ x_3 + x_4 - x_6 = 5 \\ x_i \geq 0 \quad i = 1 \dots 6 \end{cases}$$

$$\begin{aligned} & \textcircled{1} b_i \geq 0 \quad i = 1 \dots 6 \\ & \textcircled{2} x_i \geq 0 \end{aligned}$$

2) Приведём к ДКЗЛП, добавив переменные:

$$\begin{cases} x_5 + 2x_1 + x_2 + x_4 = 9 \\ y_1 + x_1 + x_2 + x_3 = 7 \\ y_2 + x_3 + x_4 - x_6 = 5 \end{cases}$$

~~3) Plus simple méthode~~

~~4) * $y_1, y_2 \geq 0$ вспомогат. переменные~~

$$W(x^*) = y_1 + y_2 \rightarrow \min$$

базисные переменные: (x_5, y_1, y_2) .

Нач. СТ:

	x_1	x_2	x_3	x_4	x_6	b
x_5	<u>2</u>	1	0	1	0	<u>9</u>
y_1	1	1	1	0	0	7
y_2	0	0	<u>1</u>	1	-1	5
$W(x^*)$	-1	-1	-2	-1	1	-12

Разреш. м-м: $i^* = y_2$
 $j^* = x_3$

Swap(i^*, j^*)

	x_1	x_2	y_2	x_4	x_6	b
x_5	2	1	<u>1</u>	1	0	9
y_1	<u>1</u>	1	<u>1</u>	-1	1	2
x_3	0	0	1	1	-1	5
	-1	-1	<u>1</u>	1	-1	-2

$i^* = y_1$
 $j^* = x_1$

	x_1	x_2	x_4	x_6	b
x_5	<u>2</u>	-1	3	-2	5
x_1	1	1	-1	1	2
x_3	<u>1</u>	0	+1	-1	5
	<u>1</u>	0	0	0	0

Получили $W(x^*) = 0 + y_1 + y_2$, где $y_1 + y_2$ — не базисные
 переменные. задано решение.

$$\begin{cases} x_5 - x_2 + 3x_4 - 2x_6 = 5 \\ x_1 + x_2 - x_4 + x_6 = 2 \\ x_3 + x_4 - x_6 = 5 \end{cases} \Leftrightarrow \begin{cases} x_5 = 5 + x_2 - 3x_4 + 2x_6 \\ x_1 = 2 - x_2 + x_4 - x_6 \\ x_3 = 5 - x_4 + x_6 \end{cases}$$

$$W(x) = 4x_1 + x_2 + x_3 + 2x_4 =$$

$$4(2 - x_2 + x_4 - x_6) + x_2 + 5 - x_4 + x_6 + 2x_4 =$$

$$13 - 3x_2 + 5x_4 - 3x_6$$

Нач. с.т.:

	x_2	x_4	x_6	b
x_5	-1	3	-2	5
x_1	1	-1	1	2
x_3	0	1	-1	5
	-3	5	-3	-13

step

	x_1	x_4	x_6	b
x_5	1	2	-1	2
x_2	1	-1	1	2
x_3	0	1	-1	5
	3	2	0	7

~~$x_1 = x_4 = x_6 = 0$~~
 ~~$x_2 = 2, x_5 = 7$~~

~~$W(x) = 3x_1 + 3x_2 + 2x_4$~~
 ~~$x_1 + x_2 - x_4 + x_6 = 2$~~
 ~~$x_3 + x_4 - x_6 = 5$~~

базис: (x_5, x_2, x_3)

$$x_4 = x_1 = x_6 = 0 \Rightarrow x_5 = 7, x_2 = 2, x_3 = 5$$

Итого получаем базисное решение: $(x_1, x_2, x_3, x_4) = (0, 2, 5, 0)$.

Ответ: $x = (0, 2, 5, 0)$, $W(x) = 7$

Ответ при решении с помощью "поиск решения" в Excel, совпадающий с ручным результатом:

A	B	C	D	E	F	G	H	I
	c	4	1	1	2	7		
	x	0	2	5	0			
		2	1	0	4	2	10	
		1	1	1	0	7	7	
		0	0	1	1	5	5	

Также был написан код на питон:

Пример входных данных:

```
min
4
4 1 1 2
3
2 1 0 1 <= 9
1 1 1 0 = 7
0 0 1 1 >= 5
```

И результат:

```
=====
РЕЗУЛЬТАТ:
Вектор x: [0.0, 2.0, 5.0, 0.0]
Оптимальное значение F: 7.0
=====
Process finished with exit code 0
```

Исходный код:

```
def read_file(file_path: str) -> tuple:
    with open(file_path, 'r', encoding='UTF8') as file:
        task_type = file.readline().strip()
        n = int(file.readline().strip())
        if n <= 0:
            raise ValueError("invalid data: n must be positive")
        c = file.readline().split()
```

```

        if len(c) < n:
            raise ValueError("invalid data: недостаточно коэффициентов c")

        if task_type not in ('max', 'min'):
            raise ValueError("invalid type: task_type должен быть 'max' или 'min'")

    m = int(file.readline().strip())
    if m <= 0:
        raise ValueError("invalid data: m must be positive")

    data = file.readlines()
    if len(data) < m:
        raise ValueError("invalid data: недостаточно строк ограничений")

    return n, c, m, data, task_type

def to_canon(n, c, m, data, task_type) -> tuple:
    if task_type == 'max':
        c = [-1 * float(k) for k in c]
    else:
        c = [float(k) for k in c]

    # оптимальное кол-во добавляем искусств столбцов
    slack_count = sum(1 for row in data if row.strip().split()[n] in ('<=', '>='))
    new_n = n + slack_count
    parsed = [[0] * (new_n + 1) for _ in range(m + 1)]

    k = 0
    for i in range(m):
        list_row = data[i].strip().split()
        parsed[i][:n] = map(float, list_row[:n])
        sign = list_row[n]

        if sign == '>=':
            parsed[i][n + k] = -1
            k += 1
        elif sign == '<=':
            parsed[i][n + k] = 1
            k += 1

        parsed[i][-1] = float(list_row[-1])

        if parsed[i][-1] < 0:
            parsed[i] = [-x for x in parsed[i]]

    new_c = list(c) + [0] * (new_n + m - len(c))

    return new_n, new_c, m, parsed

```

```

def calc_new_table(n, m, a, _i, _j, is_dop=False):
    re = a[_i][_j]
    new_a = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        for j in range(n + 1):
            if j == _j and i == _i:
                if is_dop:
                    new_a[i][j] = 0
                else:
                    new_a[i][j] = 1 / re
            elif j == _j:
                if is_dop:
                    new_a[i][j] = 0
                else:
                    new_a[i][j] = -a[i][j] / re
            elif i == _i:
                new_a[i][j] = a[i][j] / re
            else:
                new_a[i][j] = a[i][j] - a[_i][j] * a[i][_j] / re
    return new_a

def to_dop_canon(n, m, a, basis=None, free=None, is_first_step=True):
    # добавляем переменные y
    if basis is None:
        basis = [i for i in range(n, n + m)]
    if free is None:
        free = [i for i in range(n)]

    # решаем доп задачу

    if is_first_step:
        # p
        a[-1][:n] = [-sum([a[i][j] for i in range(m)]) for j in range(n)]
        # q
        a[-1][n] = -sum([a[i][n] for i in range(m)])

    _i, _j = chose_re(n, m, a)

    EPS = 1e-9

    if _i == _j == -1:
        if all(abs(x) < EPS for x in a[-1]):
            print("dop task solved")
            return n, m, a, basis, free
        else:
            print("simplex is end, sol doesn't exist")
            return n, m, a, basis, free

    # следующая симплекс таблица
    new_a = calc_new_table(n, m, a, _i, _j, True)

    # фиксируем базисы для перехода к основной задаче

```

```

basis[_i], free[_j] = free[_j], basis[_i]

# print(new_a)
# print(basis, free)
# print('new round \n\n')
return to_dop_canon(n, m, new_a, basis, free, False)

def main_task(n, c, m, a, basis, free, is_first_step=True):

    if is_first_step:
        p = [0] * (n + 1)
        for j in range(n):
            val = sum(c[basis[i]] * a[i][j] for i in range(m))
            p[j] = -(val - c[free[j]])

        p[-1] = -sum(c[basis[i]] * a[i][-1] for i in range(m))
        a[-1] = p
        _i, _j = chose_re(n, m, a)
        if _i == _j == -1:
            print("simplex is end")
            return n, m, a, basis, free

        # следующая симплекс таблица
        new_a = calc_new_table(n, m, a, _i, _j)

        # фиксируем базисы для перехода к основной задаче
        basis[_i], free[_j] = free[_j], basis[_i]

        # print(new_a)
        # print(basis, free)
        # print('new round \n\n')
        return main_task(n, c, m, new_a, basis, free, False)

def chose_re(n, m, a) -> tuple:
    p = a[-1][: -1]
    b = [a[i][-1] for i in range(m)]

    min_j = min(p)
    if min_j >= 0:
        return -1, -1
    _j = p.index(min_j)
    _i = -1
    for i in range(m):
        if a[i][_j] > 0 and b[i] >= 0:
            if _i == -1:
                _i = i
            elif b[i] / a[i][_j] < b[_i] / a[_i][_j]:
                _i = i
    if _i == -1:
        return -1, -1
    return _i, _j

```

```

def print_solution(n_orig, c_orig, a, basis, task_type):
    x = [0.0] * n_orig
    for i in range(len(basis)):
        if basis[i] < n_orig:
            x[basis[i]] = a[i][-1]

    f_val = a[-1][-1]
    if task_type == 'min':
        f_val = -f_val

    # округление для устранения погрешности float
    EPS_ROUND = 9
    x = [round(v, EPS_ROUND) for v in x]
    f_val = round(f_val, EPS_ROUND)

    print("\n" + "=" * 30)
    print("РЕЗУЛЬТАТ:")
    print(f"Вектор x: {x}")
    print(f"Оптимальное значение F: {f_val}")
    print("=" * 30)

if __name__ == "__main__":
    n, c, m, data, task_type = read_file('input.txt')
    n_canon, c_canon, m, a = to_canon(n, c, m, data, task_type)
    n_canon, m, a, basis, free = to_dop_canon(n_canon, m, a)
    n_canon, m, a, basis, free = main_task(n_canon, c_canon, m, a, basis,
free)

    print_solution(n, c, a, basis, task_type)

```